Mobile Application Development

UNIT 1: ANDROID OS DESIGN AND FEATURES

THE ANDROID PLATFORM

"Andy Rubin" has been credited as the father of the Android platform. His company, was acquired by Google in 2005.Working together, OHA members including Google, began developing a non-proprietary open standard platform based upon technology developed at Android Inc. that would aim to solve the problems hindering the mobile community.The result is the Android project.

Most Android platform development is completed by Rubin's team at Google, where he acts as VP of Engineering and manages the Android platform roadmap.Google hosts the Android open source project and provides online Android documentation, tools, forums, and the Software Development Kit (SDK) for developers. All major Android news originates at Google.

What is android?

Android is called as "the first complete, open, and free mobile platform":

Complete: allows for rich application development opportunities.

Open: It is provided through open source licensing.

Free: Android applications are free to develop. Android applications can bedistributed and commercialized in a variety of ways.

Features of Android

• Free and Open Source

• Familiar and inexpensive development tools

• Freely available SDK

• Familiar Language, Familiar Development Environments

• Reasonable learning curve for developers

• Enabling development of powerful applications

• Rich, secure application integration

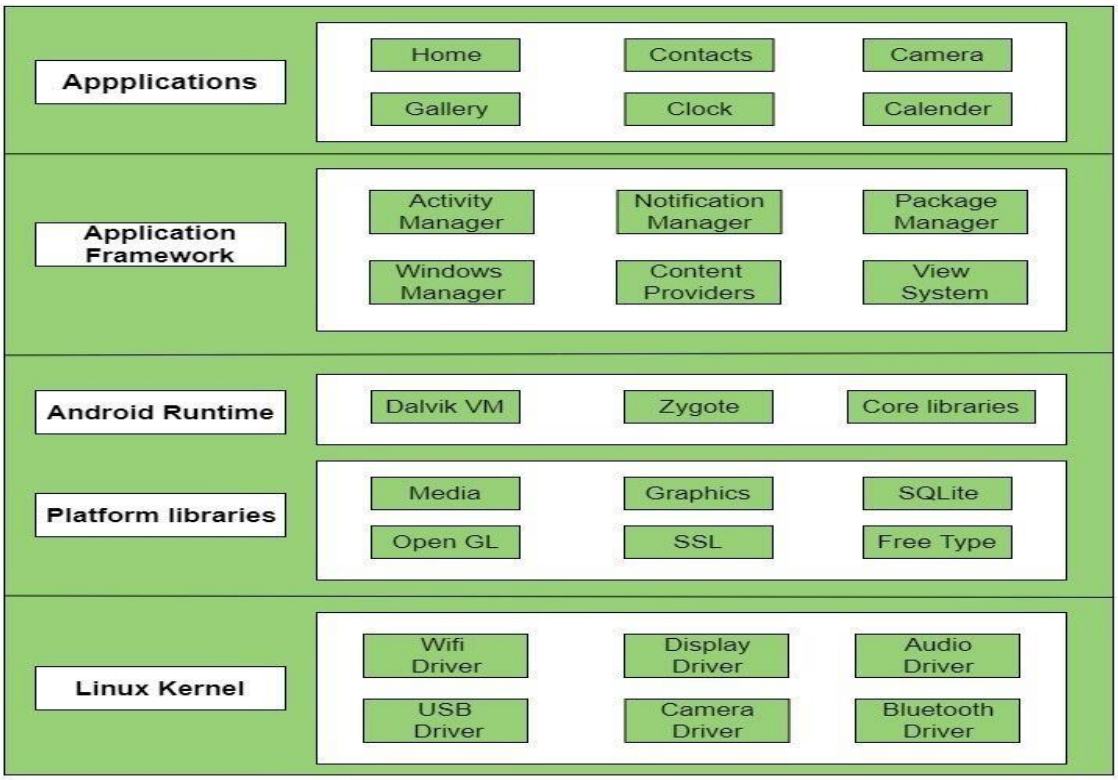
• No costly obstacles to publication

• Free "Market" for application

• A new growing platform

Android Development Framework

Android operating system is a stack of software components which is roughly divided into five sections and four main layers as shown below in the architecture diagram.



Linux kernel

At the bottom of the layers is Linux - Linux 3.6 with approximately 115 patches. This provides a level of abstraction between the device hardware and it contains all the essential hardware drivers like camera, keypad, display etc. Also, the kernel handles all the things that Linux is really good at such as networking and a vast array of device drivers, which take the pain out of interfacing to peripheral hardware.

Libraries

On top of Linux kernel there is a set of libraries including open-source Web browser engine WebKit, well known library libc, SQLite database which is a useful repository for storage and sharing of application data, libraries to play and record audio and video, SSL libraries responsible for Internet security etc.

Android Libraries

This category encompasses those Java-based libraries that are specific to Android development. Examples of libraries in this category include the application framework libraries in addition to those that facilitate user interface building, graphics drawing and database access. A summary of some key core Android libraries available to the Android developer is as follows −

- **android.app** − Provides access to the application model and is the cornerstone of all Android applications.
- **android.content** − Facilitates content access, publishing and messaging between applications and application components.
- **android.database** − Used to access data published by content providers and includes SQLite database management classes.
- **android.opengl** − A Java interface to the OpenGL ES 3D graphics rendering API.
- **android.os** − Provides applications with access to standard operating system services including messages, system services and inter-process communication.

- **android.text** − Used to render and manipulate text on a device display.
- **android.view** − The fundamental building blocks of application user interfaces.
- **android.widget** − A rich collection of pre-built user interface components such as buttons, labels, list views, layout managers, radio buttons etc.
- **android.webkit** − A set of classes intended to allow web-browsing capabilities to be built into applications.

Having covered the Java-based core libraries in the Android runtime, it is now time to turn our attention to the C/C++ based libraries contained in this layer of the Android software stack.

Android Runtime

This is the third section of the architecture and available on the second layer from the bottom. This section provides a key component called **Dalvik Virtual Machine** which is a kind of Java Virtual Machine specially designed and optimized for Android.

The Dalvik VM makes use of Linux core features like memory management and multi-threading, which is intrinsic in the Java language. The Dalvik VM enables every Android application to run in its own process, with its own instance of the Dalvik virtual machine.

The Android runtime also provides a set of core libraries which enable Android application developers to write Android applications using standard Java programming language.

Application Framework

The Application Framework layer provides many higher-level services to applications in the form of Java classes. Application developers are allowed to make use of these services in their applications.

The Android framework includes the following key services −

- **Activity Manager** − Controls all aspects of the application lifecycle and activity stack.
- **Content Providers** − Allows applications to publish and share data with other applications.
- **Resource Manager** − Provides access to non-code embedded resources such as strings, color settings and user interface layouts.
- **Notifications Manager** − Allows applications to display alerts and notifications to the user.
- **View System** − An extensible set of views used to create application user interfaces.

Applications

You will find all the Android application at the top layer. You will write your application to be installed on this layer only. Examples of such applications are Contacts Books, Browser, Games etc.

SDK Features

The Android Software Development Kit (SDK) is a collection of tools and libraries that developers use to create applications for the Android platform. Here are some of the key features of the Android SDK:

1. **Integrated Development Environment (IDE)**:
    - **Android Studio**: The official IDE for Android development, based on IntelliJ IDEA. It includes a powerful code editor, visual layout editor, and other tools to streamline development.

2. **Emulator**:
   - **Android Emulator**: Allows developers to test their applications on various Android devices and configurations without needing physical devices. It supports multiple Android versions, screen sizes, and hardware configurations.
3. **SDK Tools**:
   - **Build Tools**: Includes tools like aapt (Android Asset Packaging Tool), dx (Android Dalvik bytecode converter), and zipalign (optimizer for APK files).
   - **ADB (Android Debug Bridge)**: A versatile command-line tool that lets developers communicate with an emulator instance or connected Android device. It can be used for debugging, installing apps, and running shell commands.
4. **Libraries**:
   - **Android Support Libraries**: A set of code libraries that provide backward-compatible versions of Android framework APIs and features.
   - **Google Play Services**: Provides APIs and services for integrating Google services, such as maps, authentication, and cloud messaging, into Android apps.
5. **Development Tools**:
   - **Gradle Build System**: An advanced build toolkit that manages dependencies and automates the building, testing, and deployment of Android apps.
   - **Lint Tools**: Static code analysis tools that help identify and correct bugs, performance issues, and style inconsistencies.
6. **UI Development**:
   - **Layout Editor**: A visual editor in Android Studio that allows developers to design the user interface of their apps by dragging and dropping UI components.
   - **ConstraintLayout**: A flexible layout manager that helps create complex layouts with a flat view hierarchy.
7. **Testing and Debugging**:
   - **JUnit**: A framework for unit testing Android components.
   - **Espresso**: A testing framework for writing concise, reliable Android UI tests.
   - **Firebase Test Lab**: A cloud-based app testing infrastructure for running automated tests on a wide range of devices.
8. **Profiling Tools**:
   - **Android Profiler**: Tools for profiling an app's performance, including CPU, memory, and network usage, to identify bottlenecks and optimize performance.
9. **Support for Multiple Languages**:
   - **Java and Kotlin**: Primary programming languages supported by the Android SDK.
   - **C/C++**: Through the Android Native Development Kit (NDK), developers can use native languages for performance-critical parts of their apps.
10. **Documentation and Samples**:
    - Comprehensive documentation and sample projects are provided to help developers understand and use the features of the SDK effectively.

These features make the Android SDK a powerful and comprehensive toolkit for developing high-quality Android applications.

Installing And Running Applications On Android Studio

For developing application for android platform, you will require Integrated Development Environment (IDE). Android Studio is the official IDE for Android application development. Android Studio provides everything you need to start developing apps for Android, including the Android Studio IDE and the Android SDK tools. First we discuss what the system requirements for android studio are and how to install and configure android studio.

System Requirements for Android Studio

Windows

- ***Microsoft® Windows® 8/7/Vista/2003 (32 or 64-bit)***
- ***GB RAM minimum, 4 GB RAM recommended***
- ***400 MB hard disk space***
- ***At least 1 GB for Android SDK, emulator system images, and caches***
- ***1280 x 800 minimum screen resolution***
- ***Java Development Kit (JDK) 7***

Steps to Install Android Studio on Windows

Step 1: Head over to **this link** to get the Android Studio executable or zip file.

Step 2: Click on the **Download Android Studio** Button.

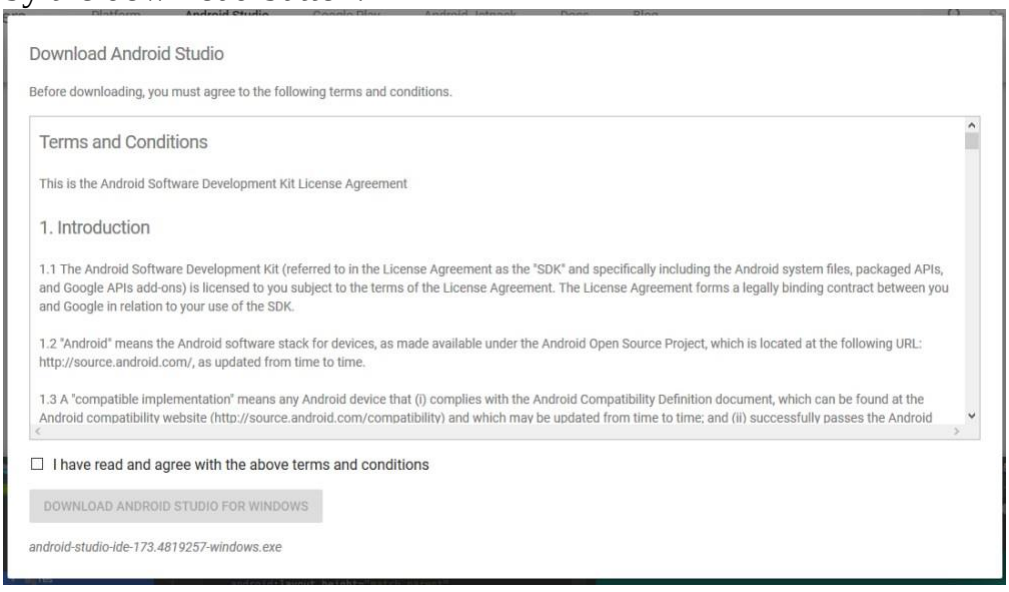


Android Studio provides the fastest tools for building apps on every type of Android device.

DOWNLOAD ANDROID STUDIO

4.1.3 for Windows 64-bit (896 MiB)

Click on the "I have read and agree with the above terms and conditions" checkbox followed by the download button.



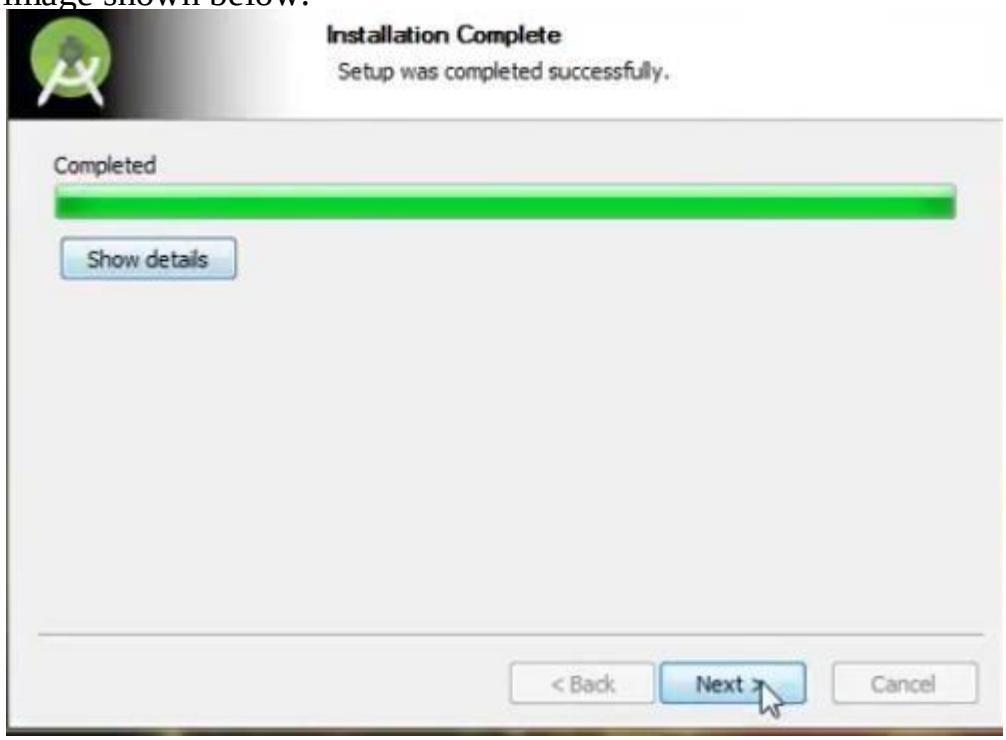
Click on the Save file button in the appeared prompt box and the file will start downloading.

Step 3: After the downloading has finished, open the file from downloads and run it. It will prompt the following dialog box.



Click on next. In the next prompt, it'll ask for a path for installation. Choose a path and hit next.

Step 4: It will start the installation, and once it is completed, it will be like the image shown below.



Click on next.



Step 5: Once "**Finish**" is clicked, it will ask whether the previous settings need to be imported [if the android studio had been installed earlier], or not. It is better to choose the 'Don't import Settings option'.



Click the **OK** button.

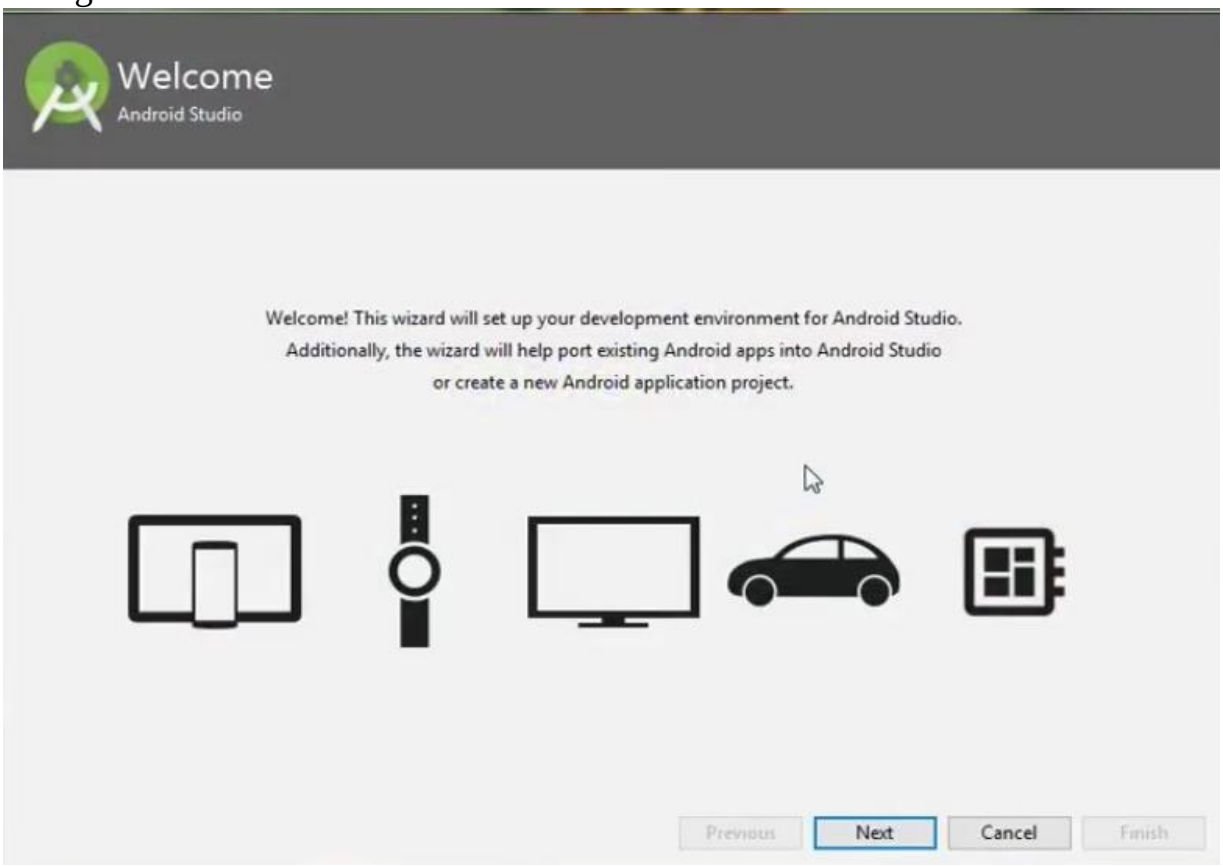
Step 6: This will start the Android Studio.



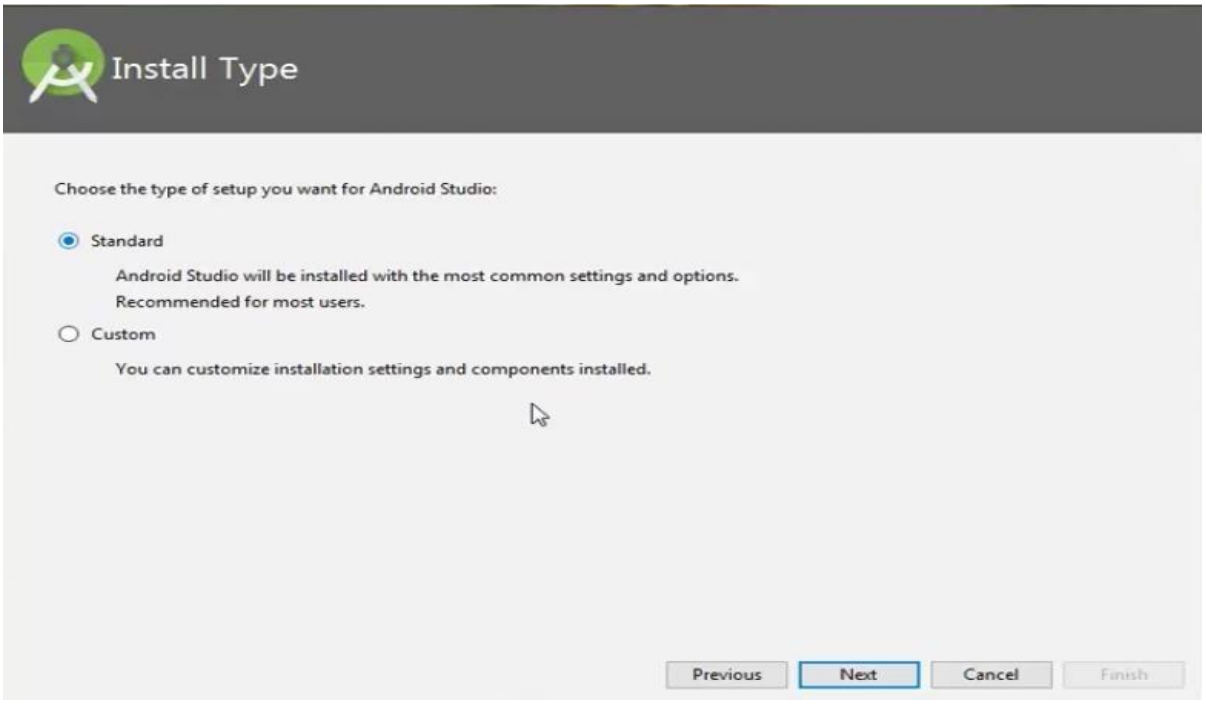
Meanwhile, it will be finding the available SDK components.



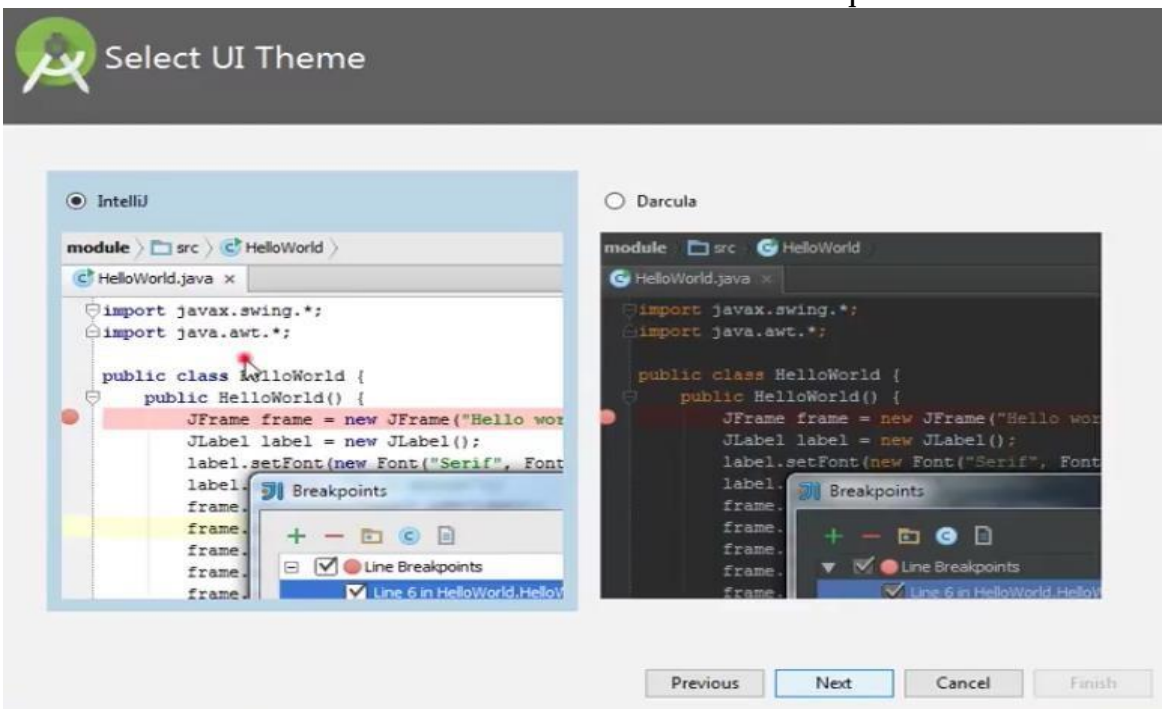
Step 7: After it has found the SDK components, it will redirect to the Welcome dialog box.



Click on **Next**.

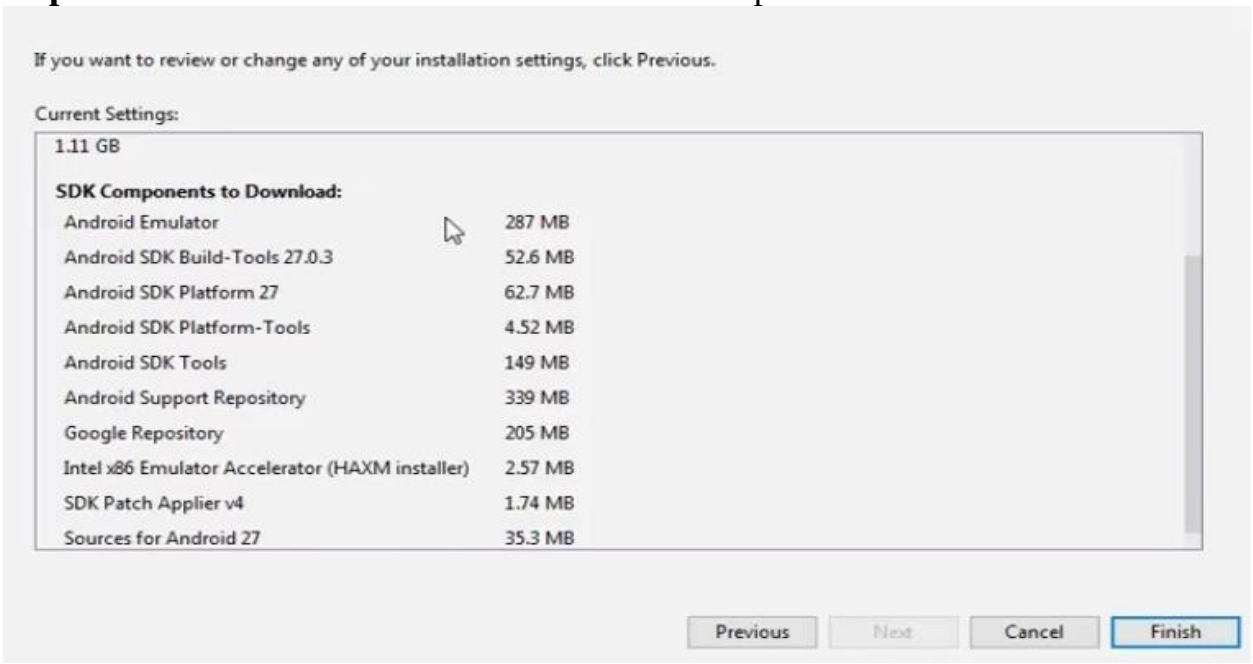


Choose Standard and click on Next. Now choose the theme, whether the **Light** theme or the **Dark** one. The light one is called the **IntelliJ** theme whereas the dark theme is called **Dracula**. Choose as required.

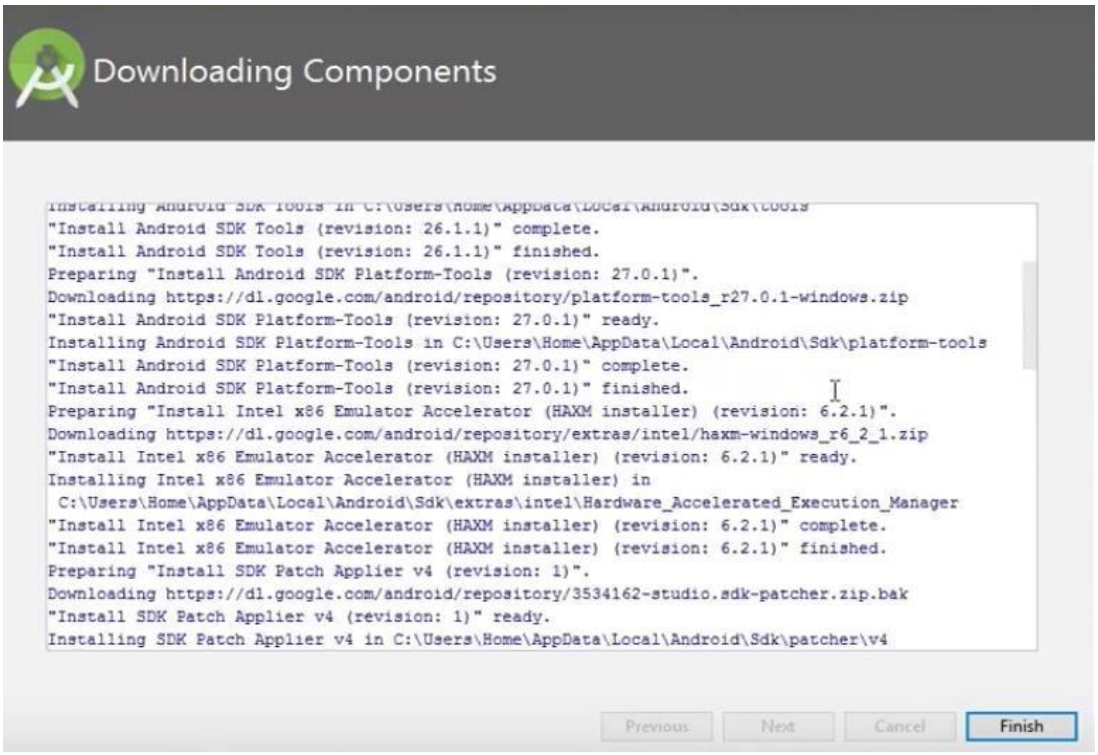


Click on the **Next** button.

Step 8: Now it is time to download the SDK components.

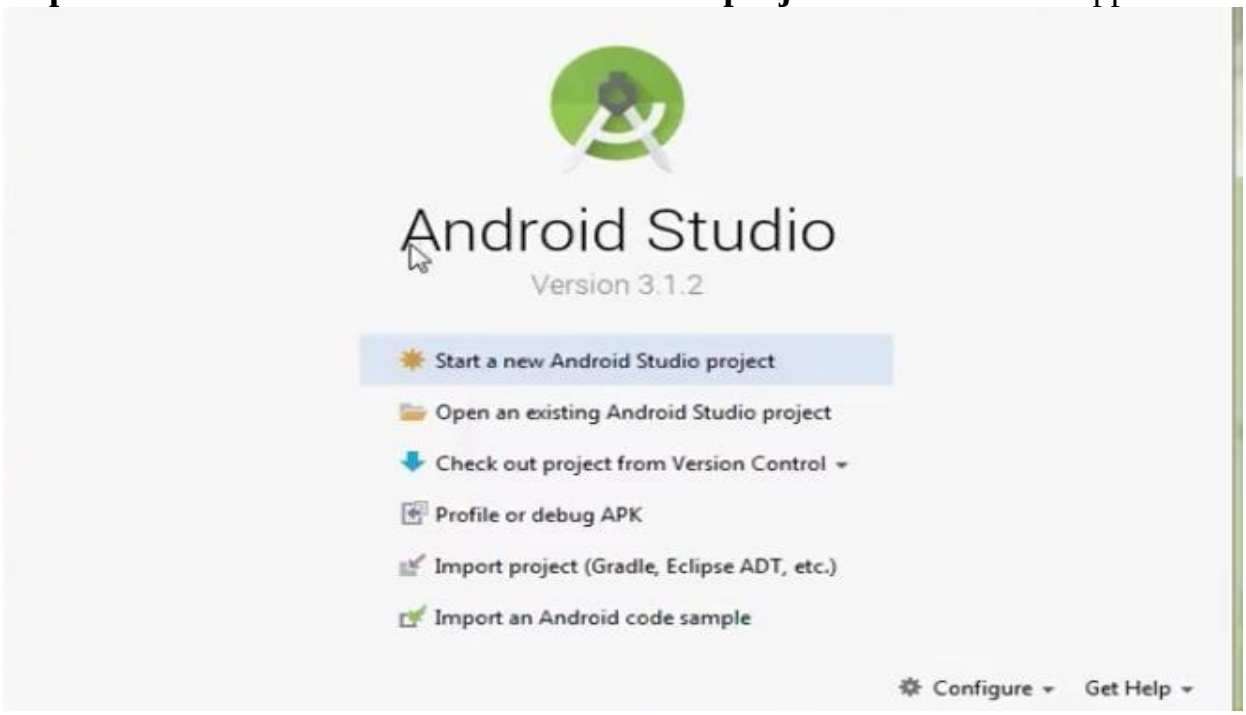


Click on Finish. Components begin to download let it complete.



The Android Studio has been successfully configured. Now it's time to launch and build apps. Click on the Finish button to launch it.

Step 9: Click on **Start a new Android Studio project** to build a new app.



To run your first android app in Android Studio you may refer to Running your first Android app.

Creating AVD's

Creating Android Virtual Devices (AVDs) in Android Studio involves setting up emulators to run and test your Android applications. Here's a step-by-step guide to help you create AVDs:

Step-by-Step Guide:

Step 1: Install Android Studio

1. **Download and Install Android Studio**:
   - Go to the Android Studio download page.
   - Download the latest version of Android Studio.
   - Follow the installation instructions for your operating system.

Step 2: Launch Android Studio

1. **Open Android Studio**:
   - After installation, launch Android Studio.
   - Complete the initial setup, including SDK installation and configuring settings as prompted.

Step 3: Open AVD Manager

1. **Access AVD Manager**:
   - In Android Studio, navigate to the top toolbar.
   - Click on the **AVD Manager** icon (it looks like a phone screen).
   - Alternatively, you can go to **Tools** > **AVD Manager** from the menu.

Step 4: Create a New Virtual Device

1. **Click on "Create Virtual Device"**:
   - In the AVD Manager window, click on the "Create Virtual Device" button.
2. **Choose a Device Definition**:
   - Select the hardware profile you want to use for your virtual device (e.g., Pixel 5, Nexus 5X).
   - Click **Next**.
3. **Select a System Image**:
   - Choose a system image (Android version) that you want to run on the virtual device.
   - You may need to download the system image if it's not already available.
   - Click **Next**.
4. **Configure AVD Settings**:
   - Name: Provide a name for your virtual device.
   - Orientation: Choose the screen orientation (Portrait or Landscape).
   - Startup Size: Configure the initial size and resolution of the emulator window.
   - Memory and Storage: Set the RAM, VM heap, and internal storage size.
   - Other Advanced Settings: Configure additional settings if needed.
5. **Finish the Setup**:
   - Click on **Finish** to create the AVD.

Step 5: Launch the AVD

1. **Start the Emulator**:
   - In the AVD Manager, you will see the list of created virtual devices.

   - Click on the green play button (triangle) next to the AVD you just created to start the emulator.
2. **Wait for the Emulator to Boot**:
   - The emulator may take a few minutes to boot up, especially the first time.
3. **Run Your Application**:
   - Once the emulator is running, you can deploy and test your Android applications on the virtual device.
   - In Android Studio, click on the **Run** button (green play icon) to build and run your application on the emulator.

By following these steps, you can create and manage Android Virtual Devices in Android Studio, allowing you to test your applications in a simulated environment

Android applications can be categorized based on their functionality, purpose, and development approach. Here are some common types of Android applications:

Types of android application

Based on Functionality and Purpose:

1. **Utility Apps**:
   - **Examples**: Calculators, Flashlights, Task Managers, Note-taking apps.
   - **Purpose**: Provide basic, essential tools and services that users might need daily.
2. **Productivity Apps**:
   - **Examples**: Office suites (e.g., Microsoft Office), Email clients, To-do lists, Calendar apps.
   - **Purpose**: Help users perform tasks efficiently and manage their work and personal schedules.
3. **Social Media Apps**:
   - **Examples**: Facebook, Instagram, Twitter, TikTok.
   - **Purpose**: Enable users to connect with friends and family, share content, and communicate.
4. **Entertainment Apps**:
   - **Examples**: Streaming services (e.g., Netflix, Spotify), Games, Video players.
   - **Purpose**: Provide entertainment through videos, music, games, and other media.
5. **Communication Apps**:
   - **Examples**: Messaging apps (e.g., WhatsApp, Telegram), Video call apps (e.g., Zoom, Skype).
   - **Purpose**: Facilitate communication through text, voice, and video.
6. **Health and Fitness Apps**:
   - **Examples**: Step counters, Workout trackers, Diet planners, Meditation apps.
   - **Purpose**: Help users monitor and improve their health and fitness.
7. **Finance Apps**:
   - **Examples**: Banking apps, Budget trackers, Investment apps (e.g., Robinhood).
   - **Purpose**: Manage personal finances, conduct transactions, and track expenses and investments.
8. **E-commerce Apps**:
   - **Examples**: Amazon, eBay, Alibaba.
   - **Purpose**: Enable users to shop online, browse products, and make purchases.
9. **Travel and Navigation Apps**:
   - **Examples**: Google Maps, Uber, Airbnb.
   - **Purpose**: Provide navigation, transportation booking, and accommodation services.
10. **Education Apps**:
    - **Examples**: Duolingo, Khan Academy, Coursera.
    - **Purpose**: Offer educational content, courses, and tools for learning new skills.

Best practices in android application

Following best practices in Android Studio can greatly improve the quality, performance, and maintainability of your Android applications. Here are some key best practices to keep in mind:

Code Organization and Structure

1. **Use Proper Package Structure**:
   - Organize your code into meaningful packages (e.g., ui, model, network, database).
   - Follow the MVVM (Model-View-ViewModel) architecture or other suitable design patterns to separate concerns.
2. **Follow Coding Standards**:
   - Adhere to the Java/Kotlin coding conventions.
   - Use meaningful variable and method names.
   - Maintain consistent code formatting with tools like Android Studio's built-in formatter or tools like ktlint.
3. **Modularize Your Project**:
   - Split your project into multiple modules (e.g., app module, feature modules, library modules) to improve build times and maintainability.

UI and UX

1. **Design for Multiple Screen Sizes**:
   - Use responsive layouts with ConstraintLayout.
   - Provide alternative resources (layouts, drawables, dimensions) for different screen sizes and densities.
2. **Follow Material Design Guidelines**:
   - Use Material Design components and themes to ensure a consistent and modern user experience.
   - Utilize the Material Components for Android library for easier implementation.
3. **Optimize for Accessibility**:
   - Provide content descriptions for UI elements.
   - Ensure proper contrast ratios and font sizes.
   - Use accessibility services to test your app.

Performance Optimization

1. **Minimize Memory Usage**:
   - Avoid memory leaks by properly managing context references.
   - Use tools like LeakCanary to detect memory leaks.
   - Recycle and reuse views and resources efficiently.
2. **Optimize Network Operations**:
   - Use efficient libraries like Retrofit for network requests.
   - Implement caching and pagination to reduce network load.
   - Use background services like WorkManager for long-running tasks.
3. **Improve App Startup Time**:
   - Delay initialization of non-critical components.
   - Use Lazy initialization for resources that are not immediately needed.
   - Optimize your app's manifest to avoid unnecessary initializations.

Code Quality and Testing

1. **Write Unit and Instrumentation Tests**:
   - Use JUnit and Mockito for unit testing.
   - Use Espresso for UI testing.
   - Aim for high test coverage and test critical paths thoroughly.
2. **Static Code Analysis**:
   - Use tools like Lint, Checkstyle, and SonarQube to analyze your code for potential issues.
   - Integrate static code analysis into your CI/CD pipeline.
3. **Code Reviews and Pair Programming**:
   - Regularly review code with peers to catch potential issues and share knowledge.
   - Practice pair programming to improve code quality and foster collaboration.

Dependency Management

1. **Use Gradle Effectively**:
   - Keep your Gradle files clean and organized.
   - Use the latest stable versions of dependencies.
   - Avoid unnecessary dependencies to reduce build times and app size.
2. **Implement Dependency Injection**:
   - Use libraries like Dagger or Hilt to manage dependencies efficiently.
   - Follow best practices for setting up and using dependency injection in your project.

Security

1. **Secure Sensitive Data**:
   - Avoid hardcoding sensitive information like API keys and secrets in your code.
   - Use secure storage mechanisms like EncryptedSharedPreferences and the Android Keystore.
2. **Implement Secure Communication**:
   - Use HTTPS for all network communication.
   - Validate SSL certificates and use certificate pinning if necessary.
3. **Protect Against Common Vulnerabilities**:
   - Regularly update your dependencies to patch known vulnerabilities.
   - Follow security best practices to prevent issues like SQL injection and XSS.

Continuous Integration and Delivery

1. **Set Up CI/CD Pipelines**:
   - Use tools like Jenkins, CircleCI, or GitHub Actions to automate builds, tests, and deployments.
   - Configure pipelines to run unit and UI tests on every commit.
2. **Automate Releases**:
   - Use Fastlane or similar tools to automate the release process to Google Play.
   - Maintain different build variants for development, staging, and production.

Documentation

1. **Document Your Code**:
   - Write clear and concise comments and Javadoc/KDoc for complex logic and public APIs.

- Maintain a well-organized README file with setup instructions and project details.

2. **Create Developer Guides**:
   - Provide internal documentation for setup, deployment, and coding standards.
   - Use tools like Confluence or Markdown files in your repository for documentation.

By following these best practices, you can create high-quality, maintainable, and efficient Android applications in Android Studio.

Android tools

Android development involves a variety of tools that help developers create, test, and optimize their applications. Here's a comprehensive list of essential Android tools:

Integrated Development Environment (IDE)

1. **Android Studio**:
   - The official IDE for Android development, based on IntelliJ IDEA.
   - Provides comprehensive tools for coding, debugging, testing, and deploying Android apps.

Build Tools

2. **Gradle**:
   - Build automation tool used in Android Studio.
   - Manages dependencies, compiles code, and packages APKs.
   - Configurable through build.gradle files.
3. **Android Gradle Plugin**:
   - Works with Gradle to build Android apps.
   - Provides specific tasks for building and testing Android projects.

Testing Tools

4. **JUnit**:
   - Framework for unit testing Java and Kotlin code.
   - Used for writing and running automated tests.
5. **Espresso**:
   - Framework for writing UI tests.
   - Allows for testing the user interface of an Android application.
6. **Robolectric**:
   - Unit testing framework that allows running Android tests on the JVM without an emulator.
7. **Mockito**:
   - Framework for creating mock objects in tests.
   - Used to test the interaction between components.

Debugging and Profiling Tools

8. **Android Debug Bridge (ADB)**:
   - Command-line tool for communicating with Android devices.
   - Allows you to perform various device actions, such as installing and debugging apps.

9. **Android Device Monitor**:
    - Provides a graphical interface for monitoring devices.
    - Includes tools for logcat, file explorer, and memory analysis.
10. **Android Profiler**:
    - Integrated in Android Studio for profiling CPU, memory, network, and energy usage.
    - Helps identify performance bottlenecks and optimize app performance.
11. **LeakCanary**:
    - Library for detecting memory leaks in Android apps.
    - Automatically identifies and reports leaks.

UI Design Tools

12. **Layout Editor**:
    - Visual editor in Android Studio for designing app layouts.
    - Supports drag-and-drop of UI components and provides real-time previews.
13. **Vector Asset Studio**:
    - Tool for importing SVG images and converting them to vector drawables.
    - Helps maintain high-quality graphics at different screen sizes.
14. **Material Design Tools**:
    - Provides components and guidelines for implementing Material Design.
    - Includes the Material Components for Android library.

Building Your First Android Applications

Following are the steps involved in creating any Android application:

1. Using Eclipse, create a new project by selecting File ➪ New ➪ Android Application Project.
2. Name the Android project suitably, say *HelloWorld*.
3. In the Package Explorer (located on the left of the Eclipse IDE), expand the HelloWorld project by clicking on the various arrows displayed to the left of eachitem in the project. In the **res/layout** folder, double-click the **activity_main.xml** file. The **activity_main.xml** file defines the user interface (UI) of your application. The default view is the ***Layout view***, which lays out the activity graphically. To modify theUI, click the **activity_main.xml** tab located at the bottom.
4. Add the following code in bold to the **activity_main.xml** file.

```
<LinearLayout
xmlns:android=”http://schemas.android.com/apk/res/android”
android:orientation=”vertical” android:layout_width=”fill_parent”
android:layout_height=”fill_parent” >
<TextView
android:layout_width=”fill_parent”
android:layout_height=”wrap_content”
android:text=”@string/hello” />
</LinearLayout>
```

5. To save the changes made to your project, press Ctrl+s.

6. You are now ready to test your application on the Android Emulator. Select the project name in Eclipse and press F11. You will be asked to select a way to debug the application. Select required Android Application and click OK.

7. The Android Emulator will now be started (if the emulator is locked, you need to slidethe unlock button to unlock it first).

8. Click the Home button (the house icon in the lower-left corner above the keyboard) so that it now shows the Home screen.

9. Click the application Launcher icon to display the list of applications installed on the device. Note that the HelloWorld application is now installed in the applicationlauncher.

(Note: In Step 6, include the steps of creating AVD).

In Android, an Activity is a window that contains the user interface of your applications. An application can have zero or more activities; in this example, the application contains one activity**: MainActivity**. This **MainActivity** is the entry point of the application, which is displayed when the application is started. When you debug the application on the Android Emulator, the application is automatically installed on the emulator.